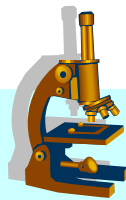


① 语言程序设计

第十讲 结构体



张 华

主要内容

- 结构体的概念
- 结构体类型的定义
- 结构体变量
 - ✱ 定义和声明
 - ✱ 初始化
 - ✱ 使用
- 结构体作函数的参数
- 自定义类型
- 结构体数组
- 程序设计举例

引例

问题

- 图书包括书号、作者、出版商、出版日期等属性。
- 怎么定义数据结构？

如何表示图书？

用多个独立的数据。

整数：书号

字符串（一维字符数组）：作者

.....

用单个数据。

图书

如何表示多本图书？

用多个并列数组。

整数数组

字符串数组（二维字符数组）

.....

用一个数组。

图书数组

简介

结构体

- ✿ 相关数据的集合。
- ✿ 数据的类型可以不相同。
- ✿ 用来定义保存在文件中的记录。
- ✿ 与指针一起创建动态的数据结构：
 - 链表
 - 队列
 - 栈
 - 树

结构体类型的定义

■ 结构体是派生的数据类型（构造类型）

- ✱ 使用其他类型的对象来构造结构体。

■ 结构体定义举例

```
/*表示纸牌*/  
struct card {  
    char *face;  
    char *suit;  
};
```



为程序创建了一个新的数据类型

struct card

- ✱ **struct**: 引入结构体定义。
- ✱ **card**: 结构体的名称，必须与 **struct** 一起使用。
- ✱ **struct card** 结构体包含两个 **char *** 类型的成员：
 - **face**
 - **suit**

结构体类型的定义

结构体定义说明

- ✱ 同一个结构体内不可以有同名的成员。
- ✱ 不同结构体的成员名可以相同，不互相冲突。

```
▶ struct date { int year,month,day; };  
▶ struct Book {  
    char title[50],writer[20],publisher[50];  
    int year,month;  
};  
▶ int year,month,day;
```

结构体类型的定义

结构体定义说明

- 结构体的成员可以是基本类型和构造类型（数组和其他结构体）。

```
struct date {  
    int year, month, day;  
};  
  
struct StuRec {  
    int num;  
    char name[20];  
    struct date birthday;  
};
```

结构体类型的定义

结构体定义说明

- ✱ 结构体不能包含自身的实例。
- ✱ 但可以包含指向自身的指针。

```
struct student {  
    char name[20];  
    char gender;  
    float scores[4];
```

```
✱ struct student next;      /*error*/  
✱ struct student *nextPtr; /*correct*/  
};
```


结构体变量

结构体定义说明

- ✿ 只是创建了新的数据类型，并不能获得内存空间。
- ✿ 必须定义结构体变量来获得内存空间。

定义声明结构体变量

- ✿ 定义结构体类型后，像声明普通变量一样声明结构体变量。

```
struct date {  
    int year, month, day;  
};  
  
struct date birth;
```

FF00

year

birth

FF04

month

FF08

day

birth 的存储结构

结构体变量

定义声明结构体变量

- ✿ 在定义结构体类型的同时，声明结构体变量

```
struct date {  
    int year, month, day;  
} birth, days[4], *bPtr;
```

- ✿ 直接（只）声明结构体变量

```
struct {  
    int year, month, day;  
} birth, days[4], *bPtr;
```

没有结构体名，
无法再次使用。

结构体的操作

在结构体（变量）上可以执行的操作

- ✿ 将结构体变量赋给相同类型的结构体变量。
- ✿ 得到结构体变量的地址。
- ✿ 访问结构体变量的成员。
- ✿ 使用 **sizeof** 确定结构体变量的大小。

结构体变量的初始化

初始化结构体变量

✿ 给全部成员赋初值。

```
struct StuRec {  
    int num;  
    char name[20];  
    struct date { int year, month, day; } birthday;  
    float score;  
} student={101, "WangHai", 1982, 5, 21, 80};
```

num (4B)	name (20B)	birthday (12B)			score (4B)
		year	month	day	
101	WangHai	1982	5	21	80.0

结构体变量的初始化

初始化结构体变量

✿ 给部分成员赋初值。

```
struct StuRec {  
    int num;  
    char name[20];  
    struct date { int year, month, day; } birthday;  
    float score;  
} student={101, "WangHai"};
```

num (4B)	name (20B)	birthday (12B)			score (4B)
		year	month	day	
101	WangHai	0	0	0	0.0

结构体变量的成员

访问结构体成员的两种方式

✿ 结构体成员运算符: .

- 用于结构体变量

```
struct card myCard;  
printf("%s", myCard.face);
```

```
struct card {  
    char *face;  
    char *suit;  
};
```

✿ 结构体指针运算符: ->

- 用于指向结构体的指针

```
struct card *cardPtr;  
printf("%s", cardPtr->face);
```

- 等价于 (*cardPtr).face

案例分析：结构体变量的成员

■ 问题：访问结构体变量的成员。

■ 实现 (cw1301.c)

```
#include <stdio.h>

struct card {
    char *face;
    char *suit;
};

void main() {
    struct card a, *aPtr;

    a.face = "Ace";
    a.suit = "Spades";
    aPtr = &a;
```

与数组的不同：
结构体变量名不是指针

案例分析：结构体变量的成员

问题

✿ 访问结构体变量的成员。

```
printf("%s%s%s\n%s%s%s\n%s%s%s\n",  
      a.face, " of ", a.suit,  
      aPtr->face, " of ", aPtr->suit,  
      (*aPtr).face, " of ", (*aPtr).suit);  
}
```

```
Ace of Spades  
Ace of Spades  
Ace of Spades
```

注意：

结构体不能作为整体输入输出。
必须逐个成员输入输出。

结构体作为函数的参数

■ 结构体的单个成员做实参

- ✿ 对应的形参是与该成员类型相同的变量。
- ✿ 被调用函数不能修改主调函数中的实参（结构体成员）。

■ 结构体变量做实参

- ✿ 对应的形参也是结构体变量。
- ✿ 被调用函数不能修改主调函数中的实参（结构体变量）。

■ 结构体变量的指针做实参

- ✿ 对应的形参是指向同类型结构体的指针变量。
- ✿ 被调用函数能修改主调用函数中的结构体。

案例分析：结构体作为函数的参数

■ 问题：编写函数实现结构体的复制。

■ 实现 (cw1302.c)

```
#include <stdio.h>

struct date {
    int year, month, day;
};

void show(char *, struct date);
void copy(struct date, struct date);
void clone(struct date, struct date *);

void main() {
    struct date d1, d2, d3, d4;
    d1.year = 2004;
    d1.month = 5;
    d1.day = 1;
    show("d1", d1);
```

案例分析：结构体作为函数的参数

源代码

✱ (续)

```
d2 = d1;  
show("d2", d2);  
  
copy(d1, d3);  
show("d3", d3);  
  
clone(d1, &d4);  
show("d4", d4);  
}  
  
void show(char *name, struct date d) {  
    printf("%s: %d-%d-%d\n", name, d.year, d.month, d.day);  
}
```

d1: 2004-5-1
d2: 2004-5-1
d3: 0-0-24
d4: 2004-5-1

案例分析：结构体作为函数的参数

源代码

✿ (续)

```
void copy(struct date s, struct date d) {  
    d = s;  
}  
  
void clone(struct date s, struct date *dPtr) {  
    *dPtr = s;  
}
```

```
d1: 2004-5-1  
d2: 2004-5-1  
d3: 0-0-24  
d4: 2004-5-1
```

在函数中使用结构体

■ 把整个结构体作为单个数据返回

- ✿ 因为结构体变量之间可以赋值。

■ 按值传递把数组传递给函数

- ✿ 把数组作为结构体的成员，然后把结构体传递给函数。
- ✿ 被调用函数不能修改主调函数中的数组。

自定义类型

typedef

✿ 为已经定义的数据类型创建一个别名。

✿ 举例

```
typedef struct date Date;
```

➤ 创建了一个新的类型名 **Date**，它是 **struct date** 的别名。

➤ 注意：并没有创建新的类型。

✿ 可以简化程序代码，提高程序的可移植性。

```
void show(char *, Date d);  
void copy(Date s, Date d);
```

```
typedef int Integer;
```

结构体数组

结构体数组

- 数组的元素是结构体变量。
- 常用结构体来表示记录，那么结构体数组就可以表示一组记录。
- 案例分析
 - 全班 N 个学生，每个学生有学号、姓名、四门课的成绩。

学号	姓名	成绩1	成绩2	成绩3	成绩4
101	WangHai	80	78	76	81
102	ZhaoFei	68	66	71	75
.....					
130	LiRui	82	76	81	84

结构体数组

结构体数组

案例分析

➤ 那么，可以定义结构体数组来保存 N 个学生的数据。

```
struct student {  
    int num;  
    char name[20];  
    float scores[4];  
};  
  
struct student students[30];
```

➤ 这样，每个学生的数据就对应一个结构体变量（一条记录），便于编程处理。

案例分析：结构体数组

高性能的洗牌和发牌程序

◆ 数据结构

- 用一个纸牌结构体数组保存一副牌。
- 纸牌的花色和号码名依然保存在字符串数组中。

◆ 这样，数组中的纸牌俨然已有一个顺序了，则可以改进算法。

- 洗牌：随机打乱纸牌在数组中的位置。
 - 不存在无限延期。
- 发牌：按纸牌在数组中的顺序显示输出。
 - 数组遍历一次。

案例分析：结构体数组

高性能的洗牌和发牌程序

✿ 实现 (cw1303.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

struct card {
    char *face;
    char *suit;
};

typedef struct card Card;

void fillDeck(Card*, char*[], char*[]);
void shuffle(Card*);
void deal(Card*);
```

案例分析：结构体数组

高性能的洗牌和发牌程序

实现

```
void main() {  
    Card deck[52];  
    char *face[] = {"Ace", "Deuce", "Three",  
                    "Four", "Five",  
                    "Six", "Seven", "Eight",  
                    "Nine", "Ten",  
                    "Jack", "Queen", "King"};  
    char *suit[] = {"Hearts", "Diamonds", "Clubs", "Spades"};  
  
    srand(time(NULL));  
  
    fillDeck(deck, face, suit);  
    shuffle(deck);  
    deal(deck);  
}
```

案例分析：结构体数组

高性能的洗牌和发牌程序

实现

```
void fillDeck(Card *wDeck, char *wFace[], char *wSuit[]) {
    int i;

    for (i=0; i<=51; i++) {
        wDeck[i].face = wFace[i%13];
        wDeck[i].suit = wSuit[i/13];
    }
}
```

案例分析：结构体数组

高性能的洗牌和发牌程序

实现

```
void shuffle(Card *wDeck) {  
    int i, j;  
    Card temp;  
  
    for (i=0; i<=51; i++) {  
        j = rand()%52;  
        temp = wDeck[i];  
        wDeck[i] = wDeck[j];  
        wDeck[j] = temp;  
    }  
}
```

案例分析：结构体数组

高性能的洗牌和发牌程序

实现

```
void deal(Card *wDeck) {  
    int i;  
  
    for (i=0; i<=51; i++) {  
        printf("%5s of %-8s%c",  
            wDeck[i].face,  
            wDeck[i].suit,  
            (i+1)%2 ? '\t' : '\n');  
    }  
}
```

案例分析：结构体数组

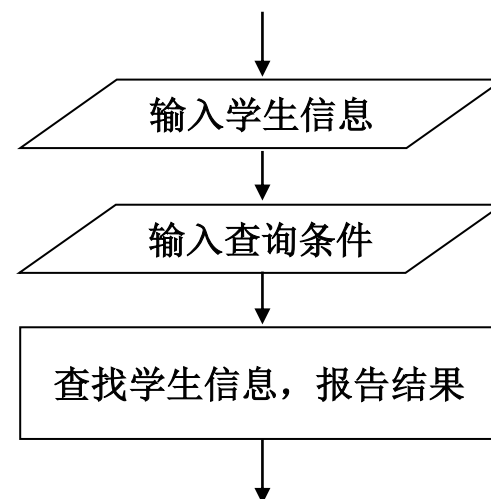
检索

问题：

- 某班有 n 个学生，每个学生的数据包括学号、姓名、年龄和性别。
- 要求给定任意一个学号，程序能输出检索的结果，并显示对应的学生的数据。

分析

- 定义结构体表示学生
- 用结构体数组保存学生数据
- 采用顺序查找法



案例分析：结构体数组

检索

实现 (cw1304.c)

```
#include <stdio.h>
#define MAX 20
void main() {
    struct StuRec {
        int num;
        char name[20];
        char gender;
        int age;
    } student[MAX];
    int i, N, num;
```


案例分析：结构体数组

检索

实现

```
printf("\tInput a integer as the number of students:");
scanf("%d", &N);

printf("\tInput %d students' information:\n",N);
printf("\n\tNo.\tName\tGender\tAge\n");
for (i=0;i<N;i++) {
    printf("student %d:\n\t", i+1);
    scanf("%d %s %c %d",
          &student[i].num,
          student[i].name,
          &student[i].gender,
          &student[i].age);
}
```

案例分析：结构体数组

检索

实现

```
printf("\n\tInput a number:");  
scanf("%d", &num);
```

输入查找条件

```
printf("\n\tPlease wait. Searching...\n");
```

```
for (i=0;i<N;i++) if (num==student[i].num) break;  
if (i<N) {  
    printf("\n\tNo.: %d\n\tName: %s\n\tGender: %c\n\tAge: %d\n",  
        student[i].num,  
        student[i].name,  
        student[i].gender,  
        student[i].age);  
}  
else  
    printf("\n\tNot found!\n");  
}
```

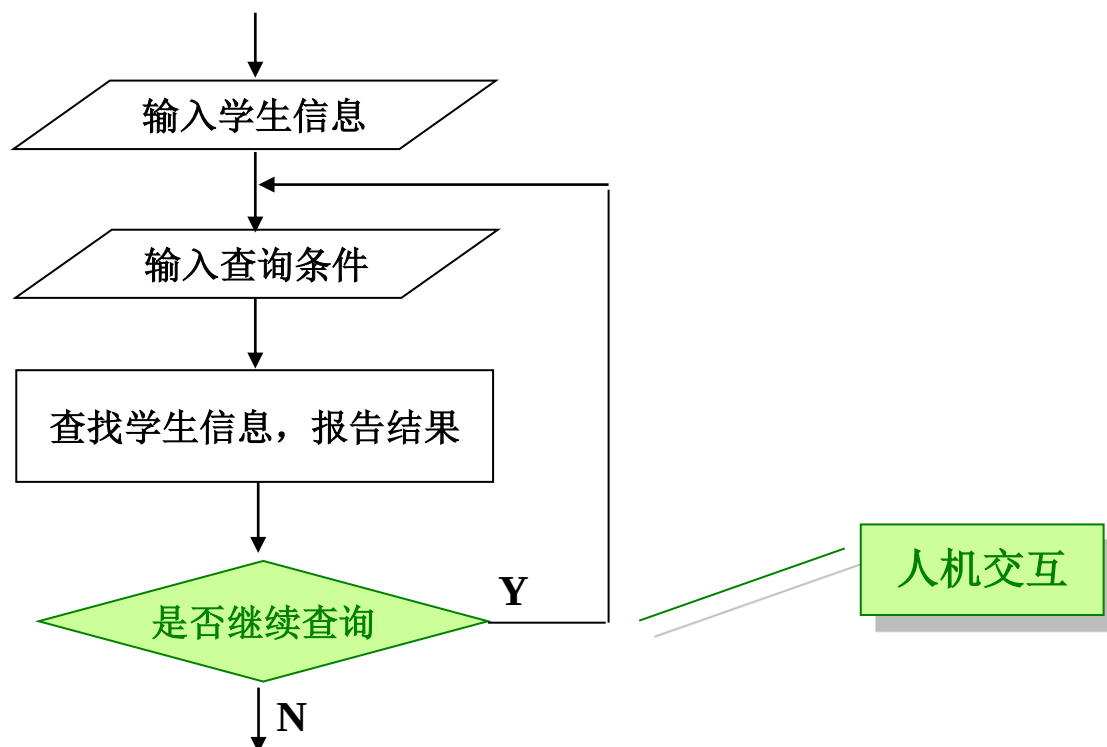
查找

案例分析：结构体数组

检索

问题：

➤ 可以多次查找。



案例分析：结构体数组

检索

✿ 实现 (cw1304m.c)

➤ 部分代码

```
do {  
    scanf("%d", &num);  
    for (i=0;i<N;i++) if (num==student[i].num) break;  
    if (i<N) { /* Found! */ }  
    else printf("\n\tNot found!\n");  
    printf("\n\tContinue?(y/n)");  
    getchar(); c=getchar();  
} while (c=='y' || c=='Y');
```

1001<Enter>y<Enter>

案例分析：结构体数组

点票程序

问题

- 有三个候选人，N个选举人，每次输入一个得票的候选人的名字，要求最后输出各人的得票结果。

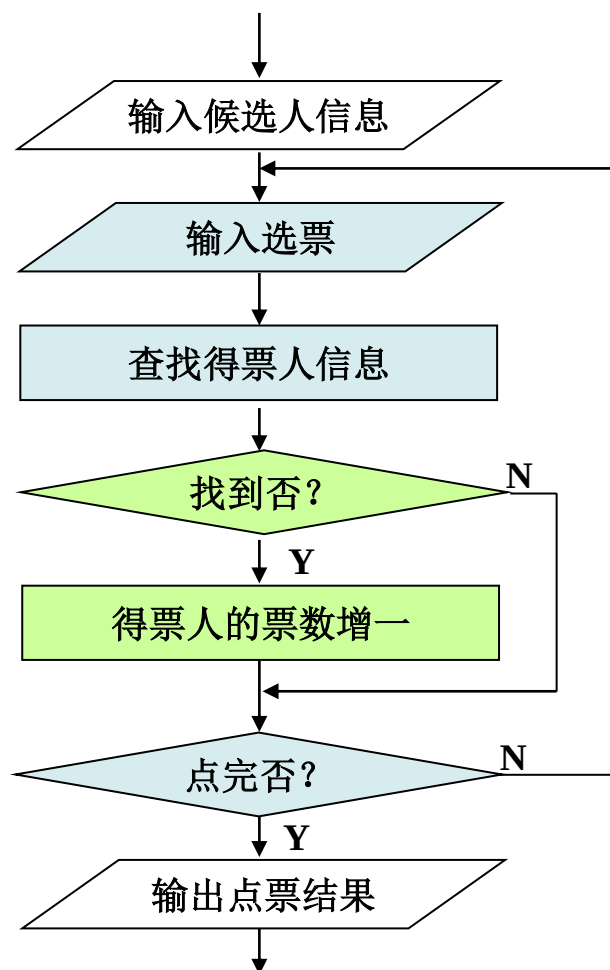
定义数据结构

```
struct candidate {  
    char name[20];    /*姓名*/  
    int count;        /*得票数*/  
} cand[3];
```

案例分析：结构体数组

点票程序

算法



案例分析：结构体数组

点票程序

✿ 实现 (cw1305.c)

```
.....  
  
do {  
    printf("Vote:\t"); gets(name); //输入选票  
    for (i=0;i<M;i++) //查找得票人  
        if (strcmp(name, cand[i].name)==0) {  
            cand[i].count++; vote++; break;  
        }  
    printf("\tContinue?(y/n)"); c=getchar(); getchar();  
} while (c=='y' || c=='Y');
```

.....

案例分析：结构体数组

改进点票程序

问题：

➤ 假设选举人都是候选人

分析点票过程中数组的变化

--	--	--	--	--	--

litao wanghai litao zhaofei

litao					
-------	--	--	--	--	--

litao	wanghai				
-------	---------	--	--	--	--

litao	wanghai	zhaofei			
-------	---------	---------	--	--	--

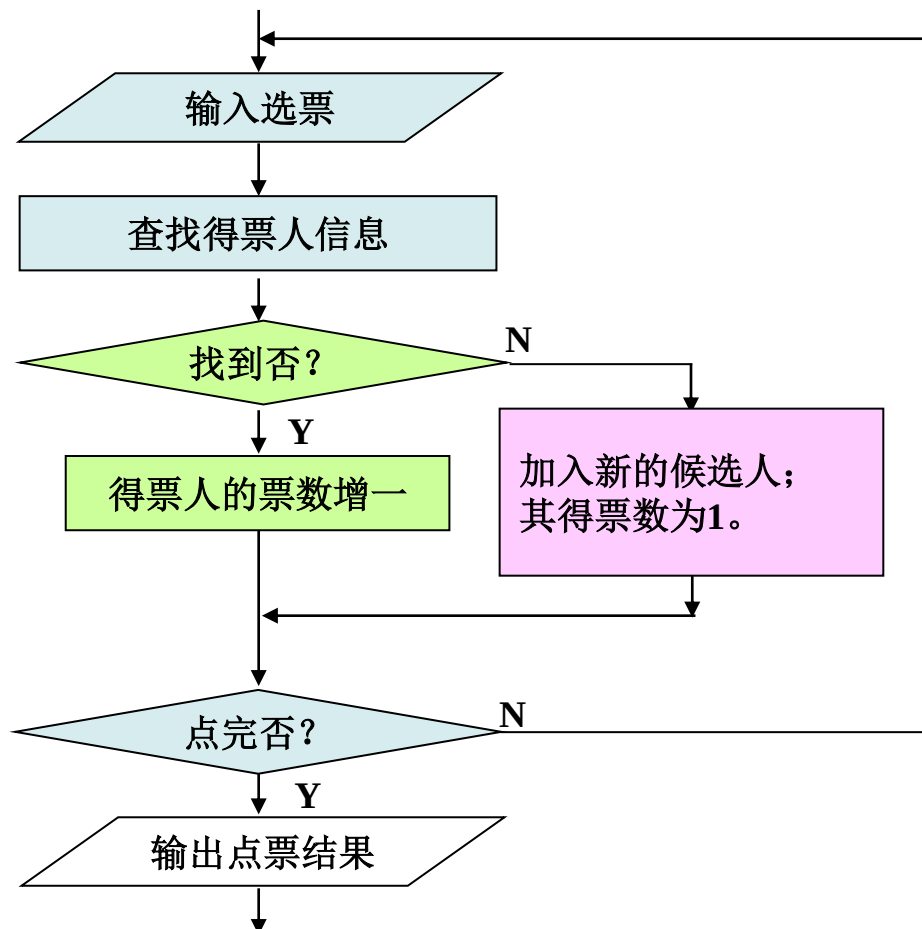


案例分析：结构体数组

改进点票程序

数据结构和算法

```
struct candidate {  
    char name[20];  
    int count;  
} cand[M];
```



案例分析：结构体数组

改进点票程序

实现 (cw1306.c)

```
...  
printf("Vote:\t"); gets(name); found=0;  
for (i=0; i<len; i++)  
    if (strcmp(name, cand[i].name)==0) {  
        cand[i].count++; found=1; break;  
    }  
  
    if (!found) {  
        strcpy(cand[i].name, name);  
        cand[i].count=1; len++;  
    }  
...
```

出现新的候选人

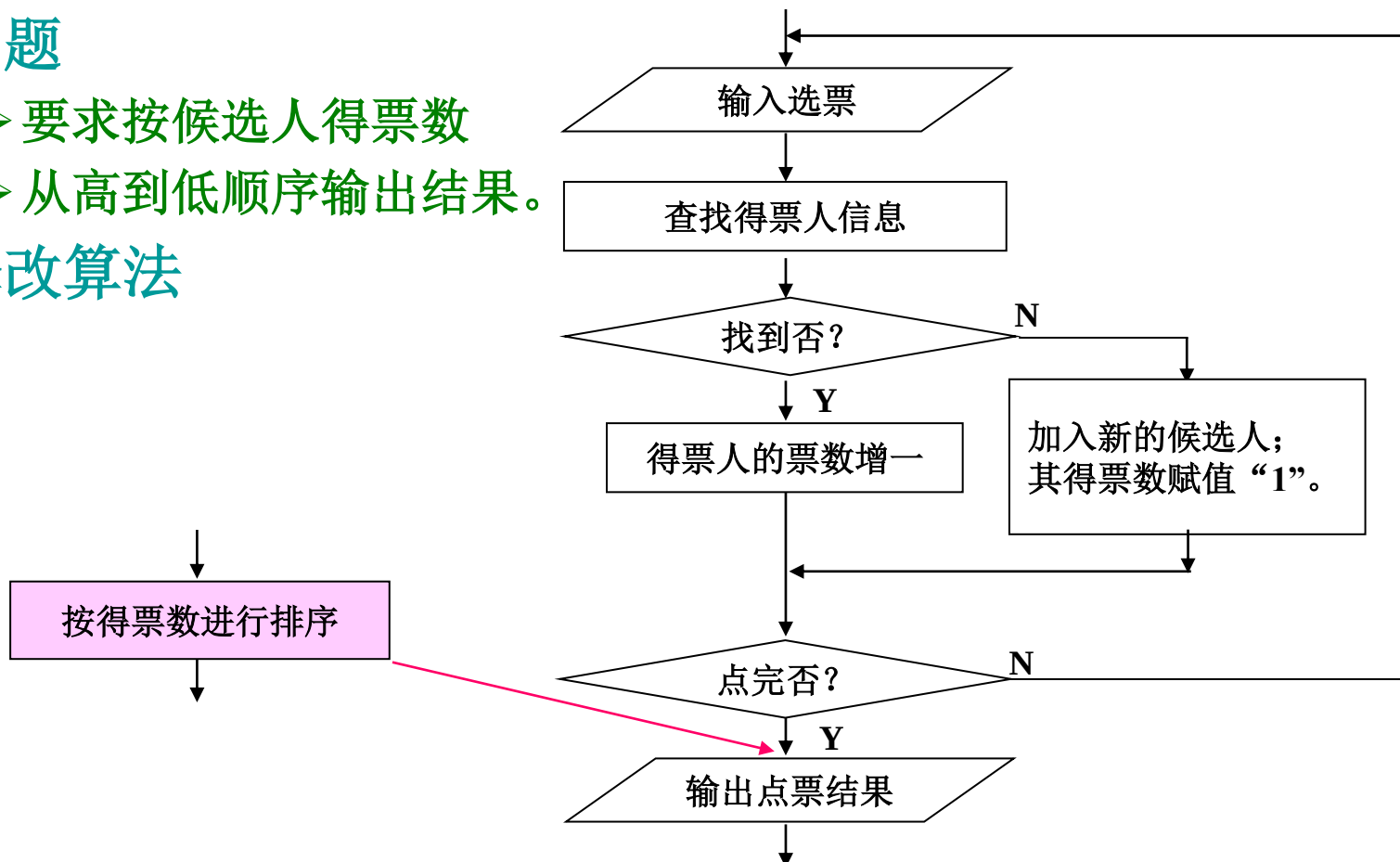
案例分析：结构体数组

增强点票程序

问题

- 要求按候选人得票数
- 从高到低顺序输出结果。

修改算法



案例分析：结构体数组

增强点票程序

✿ 实现 (cw1307.c)

➤ 排序部分

```
...  
  
    for (i=0;i<len-1;i++)  
        for (j=0;j<len-1-i;j++)  
            if (cand[j].count<cand[j+1].count) {  
                tmp=cand[j];  
                cand[j]=cand[j+1];  
                cand[j+1]=tmp;  
            }  
  
...
```

案例分析：结构体数组

优化点票程序

问题

- 如果候选人的信息较多，为了提高排序过程中数据交换的性能，增设一数组**order**，用来保存排序结果。

分析

order的初态

0	1	2	3	4	5
---	---	---	---	---	---

order的末态

3	0	5	1	4	2
---	---	---	---	---	---

从高分到低分

0	20	Wanghai M 43 P ...
1	12	Zhaofei F 41 P ...
2	6	Lilan F 38 N ...
3	35	Huangjin M 52 P ...
4	9	Wuma M 29 N ...
5	15	Hecheng M 36 P ...

案例分析：结构体数组

优化点票程序

实现 (cw1308.c)

```
for (i=0;i<len;i++) order[i]=i;
...
for (i=0;i<len-1;i++)
    for (j=0;j<len-1-i;j++)
        if (cand[order[j]].count<cand[order[j+1]].count) {
            tmp=order[j];
            order[j]=order[j+1];
            order[j+1]=tmp;
        }
```

排序数组赋初值

交换序号

案例分析：结构体数组

优化点票程序

实现

```
for (i=0;i<len;i++)  
    printf("\t%s\t%d\n", cand[order[i]].name,  
           cand[order[i]].count);
```

按照order保存的顺序输出候选人信息。

小结

- 结构体使得在一个数据中可以存储几个类型不同的数据项（成员）。
- 结构体的定义只是创建了新的数据类型，并不能获得内存空间。必须定义结构体变量来获得内存空间。
- 可以使用两种方式访问结构体数据的成员：
 - ✿ 成员运算符
 - ✿ 箭头运算符
- 结构体变量名不是地址（指针）。
- 结构体数据可以作为函数的参数，也可以作为函数的返回值。